

JavaScript Assignment

Destructuring & Array Methods — Tricky Edition

This assignment is based on the concepts you covered today: object/array destructuring, map / filter / reduce, forEach, find / some / every, includes, sort / reverse, slice / concat / join, and flat / flatMap. It is intentionally tricky — several tasks are designed to break your intuition about references, mutation, short-circuiting, and callback behavior, not just test syntax.

#	Task	File	Core Trap / Concept
1	Destructuring Under Fire	destructureTrap.js	Default values, renaming, nested & missing keys
2	The Mutation Trap	mutationTrap.js	sort()/reverse() mutate; slice() vs splice()
3	Reduce Rebuilt	reduceRebuild.js	Reimplementing map/filter using only reduce()
4	Short-Circuit Surprise	shortCircuit.js	some()/every() on empty arrays & early exits
5	Flatten the Chaos	flattenChaos.js	flat(Infinity) vs fixed depth; flatMap vs map+flat
6	Form Validator Rebuild	formValidator.js	Combining destructuring + every/some/find
7	Trick Questions (Written)	answers.js	Conceptual traps, answered as comments

Task 1 — Destructuring Under Fire

Maps to: *destructureTrap.js* | Trap: default values only apply on undefined, renaming, nested destructuring, missing keys

```
const config = {
  host: "localhost",
  port: 8080,
  db: { name: "orders", replica: null }
};

const {
  host,
  port: PORT,
  db: { name: dbName, replica = "primary" },
  timeout = 5000
} = config;

const arr = [1, 2, , 4];
const [first, second, third = 99, fourth] = arr;
```

1. Predict the exact values of PORT, dbName, replica, timeout, and third BEFORE running the code. Write your prediction as a comment, then run it and note any mismatch.
2. Explain in a comment why replica ends up as null and NOT "primary", even though a default value was provided — what is the exact rule for when destructuring defaults kick in?
3. The array arr has a hole (empty slot) at index 2. Explain what third actually becomes and why, and how that differs from a plain array index lookup arr[2].
4. Rewrite the object destructuring above to also pull out a brand-new property region that does not exist on config, giving it a default value of "ap-south-1", without changing any other line.

Task 2 — The Mutation Trap

Maps to: *mutationTrap.js* | Trap: sort() and reverse() mutate the original array in place

```
const platformArrivals = ["Chennai Exp", "Nellore Local", "Bangalore SF", "Vijayawada Pass"];

function getSortedBoard(trains) {
  return trains.sort();
}

const sortedBoard = getSortedBoard(platformArrivals);
console.log("Sorted board:", sortedBoard);
console.log("Original array:", platformArrivals);
```

1. Run the code above and observe that platformArrivals has ALSO been reordered, even though you never reassigned it directly. Explain in a comment exactly why this happens.
2. Fix getSortedBoard with the smallest possible change so the original platformArrivals array is left completely untouched, while sortedBoard still comes back alphabetically sorted.
3. Do the same experiment with reverse() on a fresh copy of the array, and in a comment state which array methods among sort, reverse, slice, splice, concat, and map mutate the original vs return a new array.
4. Using slice(), write a one-liner that safely grabs the last two arrivals from the ORIGINAL (unmutated) platformArrivals array without affecting it.

Task 3 — Reduce Rebuilt

Maps to: reduceRebuild.js | Trap: reduce() without an initial value silently changes behavior

```
const orders = [
  { id: 1, amount: 250, status: "paid" },
  { id: 2, amount: 400, status: "pending" },
  { id: 3, amount: 150, status: "paid" },
];
```

1. Using ONLY reduce() (no map, no filter), write myMap(orders, fn) and myFilter(orders, fn) helper functions that behave identically to Array.prototype.map and Array.prototype.filter.
2. Use your myFilter to get only "paid" orders, then your myMap to extract just their amount values, and finally a plain reduce() to sum them — log the final total.
3. Run reduce() on orders WITHOUT passing an initial value, using a callback that expects each element to be a number (e.g. (sum, o) => sum + o.amount). Observe the error or NaN result and explain in a comment exactly why omitting the initial value is dangerous here.
4. In a comment, state the one rule for when reduce() MUST be given an initial value, and give one example where it is safe to omit it.

Task 4 — Short-Circuit Surprise

Maps to: shortCircuit.js | Trap: every() on an empty array is always true; some() short-circuits and skips remaining checks

```
const emptyQueue = [];
const ticketPrices = [120, 85, 300, 60];
```

1. Run emptyQueue.every(price => price > 0) and emptyQueue.some(price => price > 0) and log both. Predict the two results BEFORE running, then explain in a comment why an empty array makes every() return true and some() return false — this is called "vacuous truth".
2. Write a callback for ticketPrices.some(...) that has a console.log INSIDE it, so you can see which elements were actually checked. Confirm in a comment that some() stops calling the callback the moment it finds a match, instead of checking every element.

3. Do the same experiment with `every()` — add a `console.log` inside the callback and confirm it stops as soon as ONE element fails the condition.
4. Using `find()` (not `filter()`), locate the first ticket price above 250, and in a comment explain why `find()` is a better choice than `filter()[0]` for this use case.

Task 5 — Flatten the Chaos

Maps to: flattenChaos.js | Trap: default flat() depth is only 1; flatMap only flattens one level and cannot go deeper

```
const deeplyNested = [1, [2, 3, [4, 5, [6, 7]]]];
const words = ["hello world", " javascript rocks", "flat map fun"];
```

1. Call `deeplyNested.flat()` with no arguments and log it — notice it is only PARTIALLY flattened. Explain in a comment exactly why, and what the default depth argument actually is.
2. Fix it so `deeplyNested` becomes a single flat array `[1,2,3,4,5,6,7]`, regardless of how deep the nesting goes, without hardcoding a depth number.
3. Using `flatMap()`, turn the `words` array into a single flat array of individual lowercase words with no extra whitespace (hint: split on whitespace using a regex, and `flatMap` only flattens one level — explain in a comment why that is enough here but would NOT be enough for `deeplyNested`).
4. In a comment, explain the exact difference between `arr.flatMap(fn)` and `arr.map(fn).flat()` — are they always equivalent, and is one more efficient?

Task 6 — Form Validator Rebuild

Maps to: formValidator.js | Trap: combining destructuring with every/some/find, and a field that is falsy but valid

```
const fields = [
  { name: "email", value: "a@b.com", valid: true },
  { name: "age", value: 0, valid: true },
  { name: "password", value: "", valid: false },
];
```

1. Using destructuring INSIDE the arrow function parameters (e.g. `({ valid }) => valid`), write `canSubmit`, `hasErrors`, and `firstInvalid` exactly as in today's signup-form example, but this time also destructure out `name` and `value` wherever you log them.
2. Notice the "age" field has `value: 0`. Write a naive check like `fields.every(({ value }) => value)` and observe that it wrongly reports age as "missing", even though `valid` is true. Explain in a comment exactly why 0 breaks a naive truthy check, and fix the check to rely on the `valid` flag instead of the raw value.
3. Write a one-liner using `find()` plus destructuring in the same expression to directly pull out the name of the first invalid field (or undefined if none exist) in a single statement.
4. Combine everything: build a summary object `{ canSubmit, invalidFieldNames }` where `invalidFieldNames` is built using `filter()` + `map()` chained together, and destructure `name` straight out of the callback parameters for both.

Task 7 — Trick Questions (Answer as Comments)

Maps to: answers.js | No code execution needed — answer each as a comment block

1. If you destructure `{ a, b }` from an object that is actually null or undefined, what happens, and how would you safely destructure a value that MIGHT be null without crashing?
2. Does `Array.prototype.map()` skip holes in a sparse array the same way `forEach()` does? Justify with one sentence, and name one method that does NOT skip holes.

3. You call `arr.sort()` on an array of numbers like `[10, 2, 33, 4]` with no comparator. What is the actual output, and why does `sort()` behave this way by default?
4. In array destructuring, can you skip elements you don't need? Show the syntax for skipping the 2nd element while grabbing the 1st and 3rd from `[10, 20, 30]`.
5. Is `includes()` able to correctly find `NaN` inside an array, when `indexOf()` cannot? Explain the one-sentence reason why these two methods disagree on `NaN`.

Submission note: Each numbered file above should run independently with `node filename.js` and print clearly labeled console output for every step. Where a task asks for an explanation, write it as a `//` comment directly above or below the relevant code — do not just keep it in your head.